

μFé-8

Um microcontrolador de 8 bits inspirado no MSP430. Funciona como um elo entre duas disciplinas da graduação: Sistemas Digitais e Sistemas Microprocessados. Tentei simplificar ao máximo a arquitetura do MSP430 para algo que fosse simples de entender, porém mantendo os aspectos principais da arquitetura para facilitar a transição entre SisDig e SisMic.

Apresentação da arquitetura:

O processador possui 4 registros para o usuário

- R0 a R3 são os registros de 8 bits de uso geral.

Internamente, a arquitetura conta com outros 3 registradores:

- PC: Contador do programa. É um registrador especial que incrementa +1 toda vez que for lida uma instrução ou valor imediato (constante) da memória ROM. Ele sempre contém um endereço que aponta para a próxima instrução que será executada.
- IR: Instruction Register. Segura a instrução durante todo o ciclo de execução.
- CTE: Guarda o valor da constante usada no modo imediato (explicado mais a frente)

Instruções

O conjunto de instruções é codificado em 8 bits e possui ao total 10 instruções. A sintaxe geral de uma instrução é a seguinte:

INSTR SRC, DST

Os campos SRC e DST correspondem à fonte (SRC) e o destino (DST) da operação. Cada campo pode ser escrito em duas formas, chamadas de “modos de endereçamento”:

Fonte (SRC):

- Registro Rs
- Imediato #i (valor constante)

Destino (DST):

- Registro Rd

O modo imediato é implementado internamente codificando a constante no endereço imediatamente subsequente à instrução. Assim, é necessário um ciclo adicional para buscar esse valor na ROM.

Exemplos

Para entender melhor as instruções e seus modos de endereçamento, ofereço dois exemplos: O primeiro calcula a série de Fibonacci

start:

```
MOV #1, R0    // Inicializa R0 = 1
MOV #2, R1    // Inicializa R1 = 2
```

fibo:

```
MOV R1, R2    // Faz um backup de R1 em R2
ADD R0, R1    // R1 = R1 + R0 (1+2)
MOV R2, R0    // R0 = R2
JMP fibo      // Na próxima iteração (R0,R1) = (2,3)
```

O segundo exemplo usa saltos condicionais

mult:

```
MOV #0, R0    // Limpa R0 e R1
MOV #0, R1
```

mult_loop:

```
ADD R2, R0    // Acumula R2 em R0, R3 vezes
JC  mult_c    // Esses dois saltos
JMP mult_dec  // simulam o JNC
```

mult_c:

```
ADD #1, R1    // Carry vai para MSByte
```

mult_dec:

```
ADD #-1, R3   // decrementa R3
JNZ mult_loop // até chegar em zero
```

A instrução ocupa exatamente 1 byte (8 bits) com um complemento opcional marcado pela presença do bit CTE.

Instrução							
OP			CTE	SRC		DST	
op2	op1	op0	cte	s1	s2	d1	d0

A lista completa das instruções está abaixo

```
// Operações aritméticas e lógicas
000 c ss dd | ADD    Rs/#i, Rd
001 c ss dd | AND    Rs/#i, Rd
010 c ss dd | XOR    Rs/#i, Rd
011 c ss dd | OR     Rs/#i, Rd

// Acesso à memória (Load e store)
100 c ss dd | LD     @Rs/@i, Rd
101 c ss dd | ST     Rs/#i, @Rd

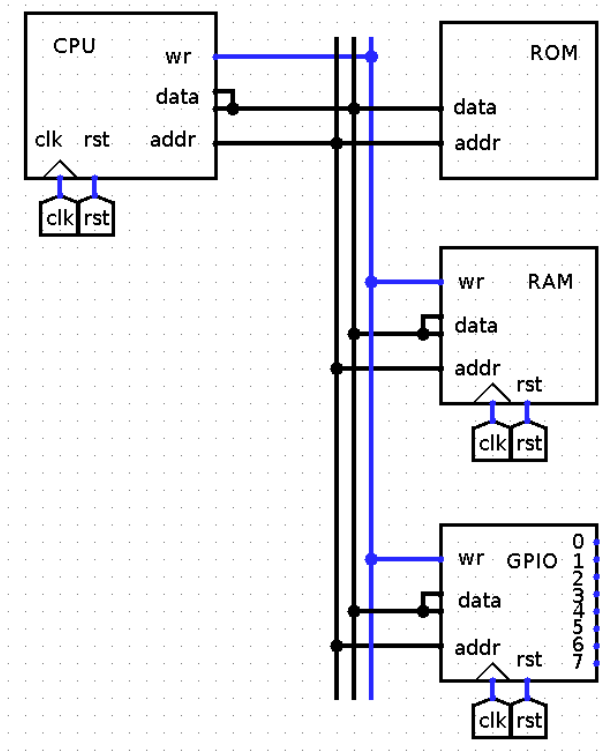
// Movimentação de dados
110 c ss dd | MOV    Rs/#i, Rd

// Saltos
111 1 xx 00 | JC     addr
111 1 xx 01 | JZ     addr
111 1 xx 10 | JNZ    addr
111 1 xx 11 | JMP     addr

OBS: Quando cte = 1, ss é don't care
```

Visão geral do sistema

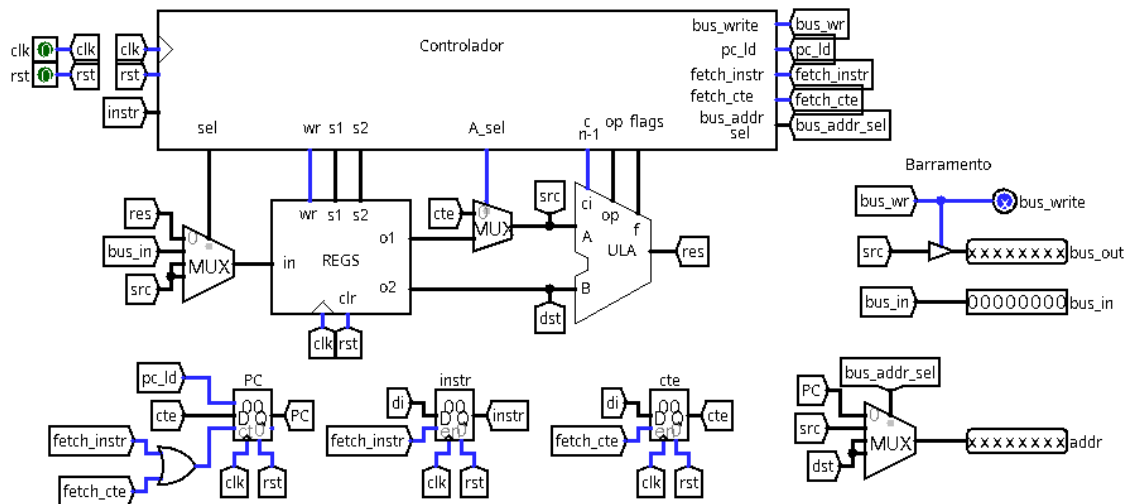
O sistema será composto por um processador, uma memória RAM para a pilha e uma memória ROM para as instruções. Além disso, podem ser conectados ao barramento, periféricos de hardware, como GPIO, Timers, ADC, etc.. Neste desafio, nós vamos fazer apenas o processador.



Mapa da memória (endereços)

Periféricos	Fim	0xFF
	Início	0xE0
RAM	Fim	0xDF
	Início	0xA0
ROM	Fim	0x9F
	Início	0x00

O processador tem apenas um banco de registradores e uma ULA que são controlados por uma máquina de estados. Os terminais de acesso da CPU são feitos pelo barramento, ou seja, dados, endereço e um sinal de escrita. A imagem abaixo é do modelo do circuito do logisim. Esse simulador não tem pinos bidirecionais, então tive que separar o barramento de dados em bus_out e bus_in, mas na prática, são os mesmos fios.



Descrição dos circuitos

A ULA é um circuito combinacional com duas entradas A e B e realiza a operação dada pelo seletor op (2 bits). Ela recebe também uma entrada adicional correspondente ao carry da operação passada. As flags geradas são apenas 2: Carry e Zero (que indica que o resultado deu zero).

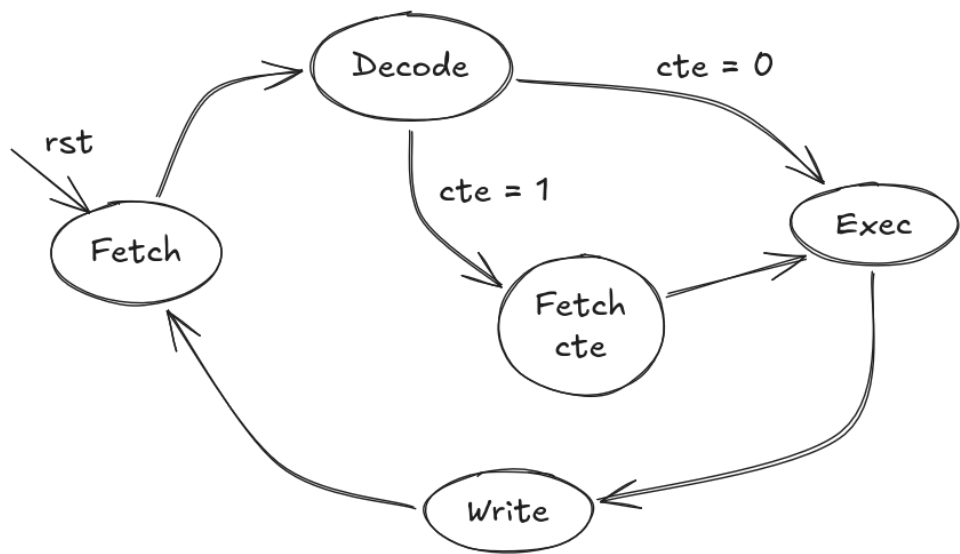
O banco de registradores (REGS) é um agrupamento dos 4 registradores, R0 a R3. O circuito conta com uma entrada que pode vir do resultado da ULA, do barramento ou do registro de fonte (SRC). Esse bloco tem duas saídas que podem ser selecionadas a partir dos seletores s1 e s2. O sinal wr controla se a entrada vai ser escrita no registrador selecionado por s2. As saídas o1 e o2 sempre estão disponíveis para leitura.

O controlador é uma máquina de estados para controlar esse caminho de dados (datapath). Essa máquina deve

- Fazer a leitura da instrução, guardar o resultado no IR,
- decodificar a instrução para verificar se há necessidade de um passo adicional para buscar a constante na ROM,
- executar a instrução e
- se houver necessidade de gravar o resultado, fazê-lo no final.

Esse ciclo se repete indefinidamente.

Abaixo um esboço da máquina de estados. Ela ainda não está toda testada e pode ser modificada para acomodar ajustes e melhorias.



Sugestão de desenvolvimento: Comece pela ULA ou pelo banco de registradores. Deixe a máquina de estados por último. Faça uma tabela com a sequência dos sinais de controle para cada estado e instrução e veja se consegue generalizar.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1								r_in	fetch	fetch			reg A			PC	bus	bus
2								sel	Inst	cte	s1	s2	wr	sel	op	ld	wri	addr
3							Fetch											
4		OP+C+SRC+DST					Decode											
5		code	c	ss	dd		Fetch cte											
6																		
7	Logic/Arith	Opp	c	ss	dd		Exec											
8							Write											
9																		
10	LD	100	c	ss	dd		Exec											
11							Write											
12																		
13	ST	101	c	ss	dd		Exec											
14							Write											
15																		
16	MOV	110	c	ss	dd		Exec											
17							Write											
18																		
19	Saltos	111	1	xx	cond		Exec											
20							Write											

A ideia é mapear as operações da ULA nos op-codes da ISA e criar caminhos de dados que permitam executar todas as instruções. Veja que as operações de movimentação de dados (MOV) não usam a ULA.